arm

Introduction to Verification

Module1 ETIN40 ICP-2



Manjiri Deolalikar
2026-08-31

© 2026 Arm

About me

Manjiri Deolalikar

Building confidence in complex digital hardware through smart verification

Education

BE, Electronics & Telecom — Pune University • 2001

PG Diploma, VLSI — Bitmapper Institute • 2003

Experience & expertise

23+ years in ASIC

Ingot Systems • Ericsson • Arm

Digital design • Verification • Project & line management • Director Engineering



Verification leadership

across Arm Neural Processing IPs

- **Ethos U55**
- **Ethos U65**
- **Ethos N78**
- Next-generation ML Accelerators

arm

© 2026 Arm

Agenda

- 🔍 What is Verification?
- ❓ Why Verification?
- ⚙️ Verification Process
- 🛠️ Various Verification Techniques
- 📋 Verification Planning
- ✅ Verification Sign-off
- 🤖 AI for Verification
- 📌 Summary



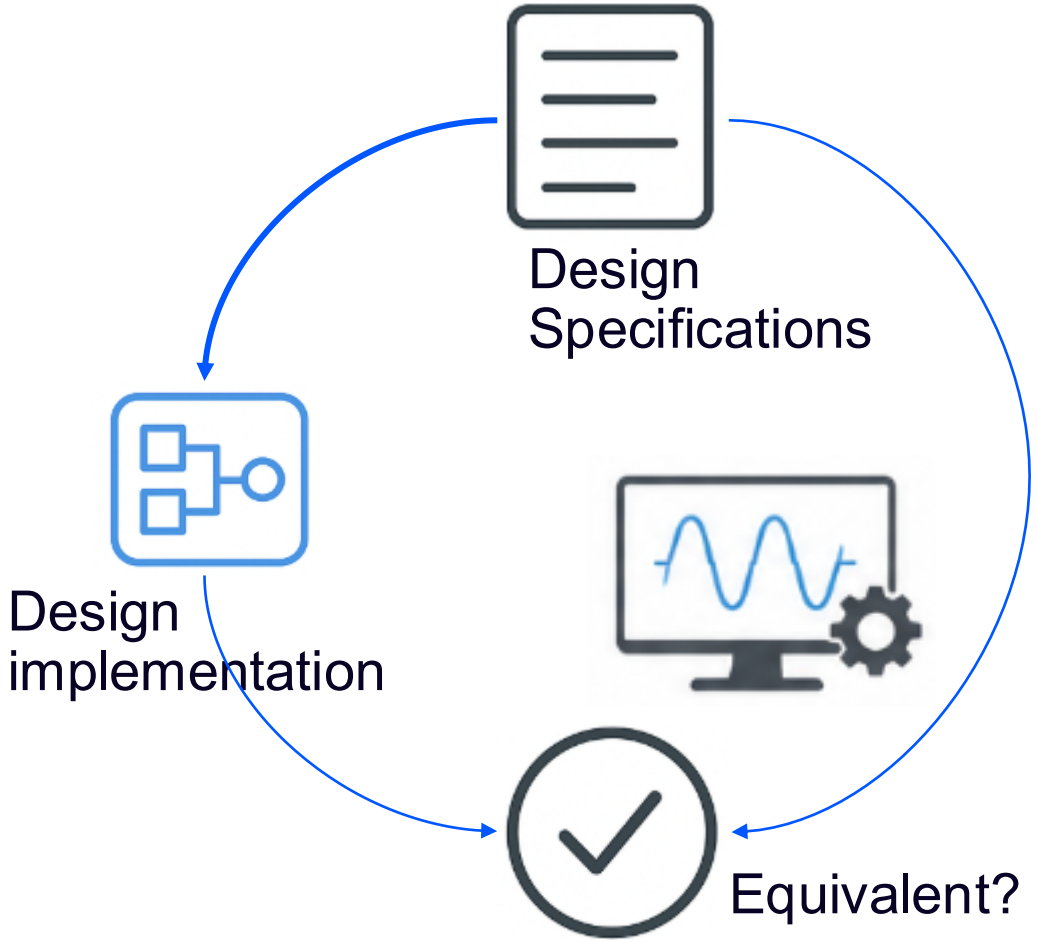
arm
© 2026 Arm

What is Verification?

© 2026 Arm

What is Verification?

- Verification is a process of demonstrating functional correctness of a design against design specifications

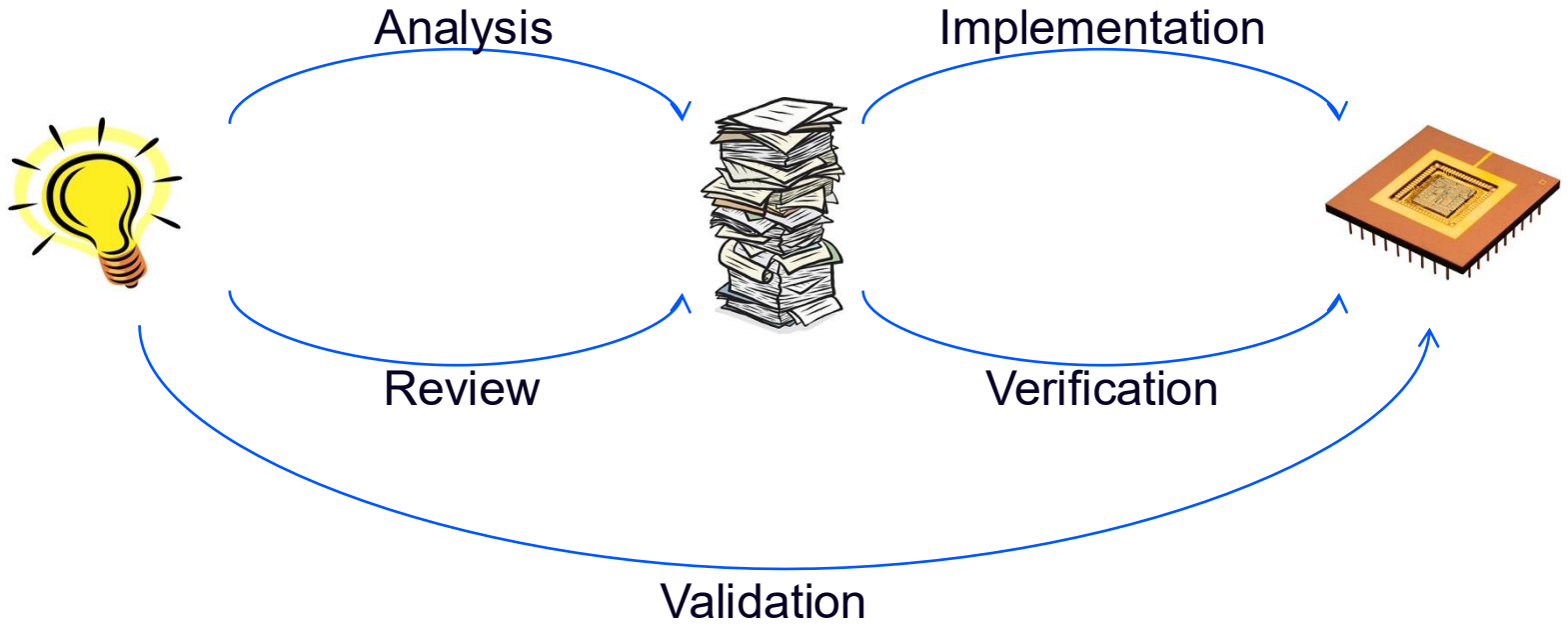


“Are we building the product right?”

- Apply constrained-random stimulus, formal methods
- Check functional correctness against expected behavior
- Measure coverage.

© 2026 Arm

Verification vs Validation



Verification (Are we building the product, **right**?)

- Pre-silicon
- Check that implementation conforms to specs
- Focus – Functional correctness, protocols, corner cases and coverage
- Methods – RTL simulation, assertions, formal verification, Reviews, Static checks

Validation (Are we building the **right** product?)

- Post-silicon / system-level
- Check that the product satisfies intended purpose
- Focus – End-to-end behavior, usability, performance
- Methods – software workloads, performance, interoperability, field/usecase testing

© 2026 Arm

Why Verification?

© 2026 Arm

Importance of Verification

WHY IT MATTERS

1. **Find bugs early**

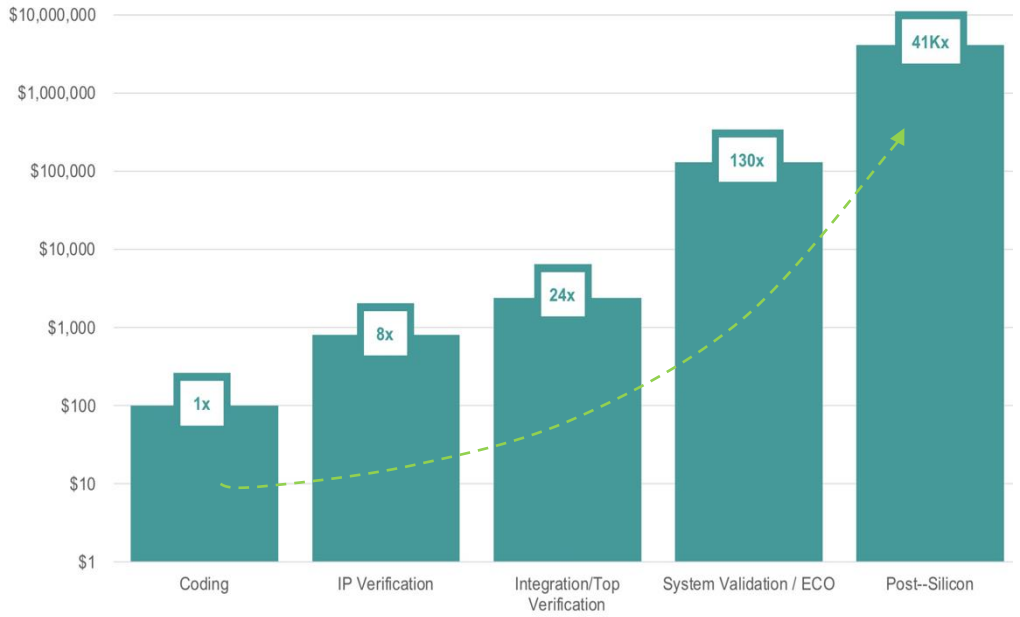
   Defects are cheaper and faster to fix before tape-out.

2. **Reduce project risk**

   Avoid re-spins, schedule slips and downstream product impact.

3. **Build tape-out confidence**

   Create measurable evidence that requirements have been verified.



Cost of finding and fixing bugs Vs Development stage

@Siemens 2021 Improving initial RTL quality, Siemens EDA

Verification lesson **A bug found in RTL costs engineering time. A bug found after fabrication can require new silicon, replacement inventory and customer recovery.**

Find bugs before tape-out — when they are still cheap to fix.

© 2026 Arm

Intel floating point division bug (1993)

- **Problem**: 0.006% error in some floating-point division calculations
  - Bug in Pentium x86 processor, found by Thomas Nicely, maths professor
  - If ((4195835 / 3145727) * 3145727 – 4195835) != 0 → Intel FDIV bug!

- **Solution**: Make key-chains for employees!
  - Intel replaced ~5 million faulty chips – costed $475 M



© 2026 Arm

ASIC industry trends - Bugs

- The percentage of “logic or functional flaws” remains the leading cause of bugs.

- Design errors are the leading cause of functional flaws
- In addition, problems associated with changing, incorrect, and incomplete specifications are a common theme

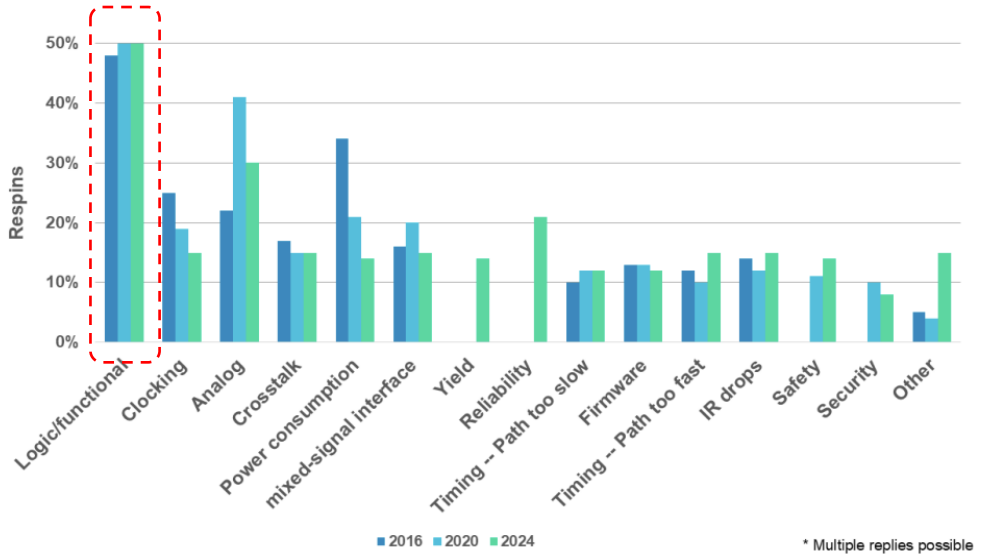


Figure 4. Types of flaws contributing to respins.

2024 Wilson Research Group IC/ASIC functional verification trend report

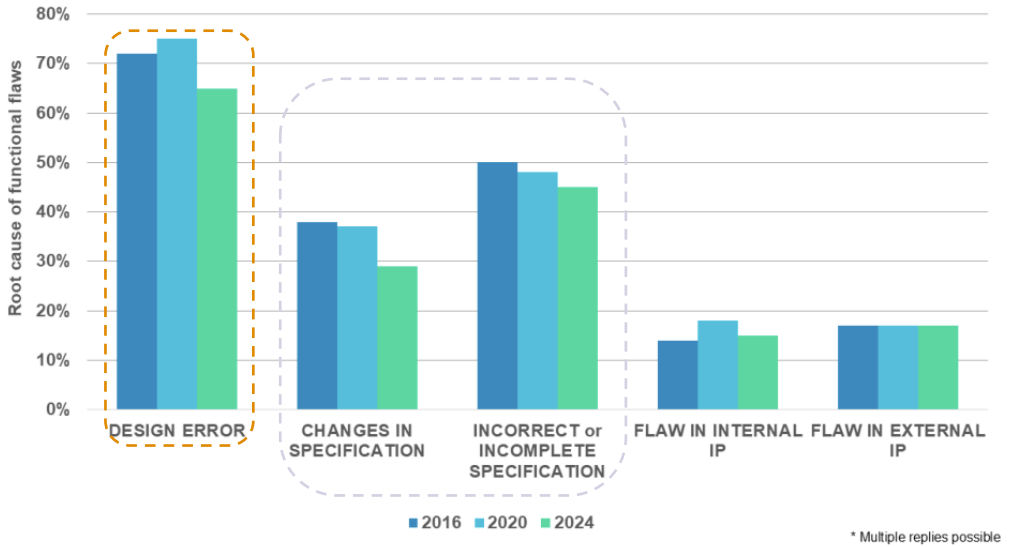


Figure 5. Root cause of functional flaws.

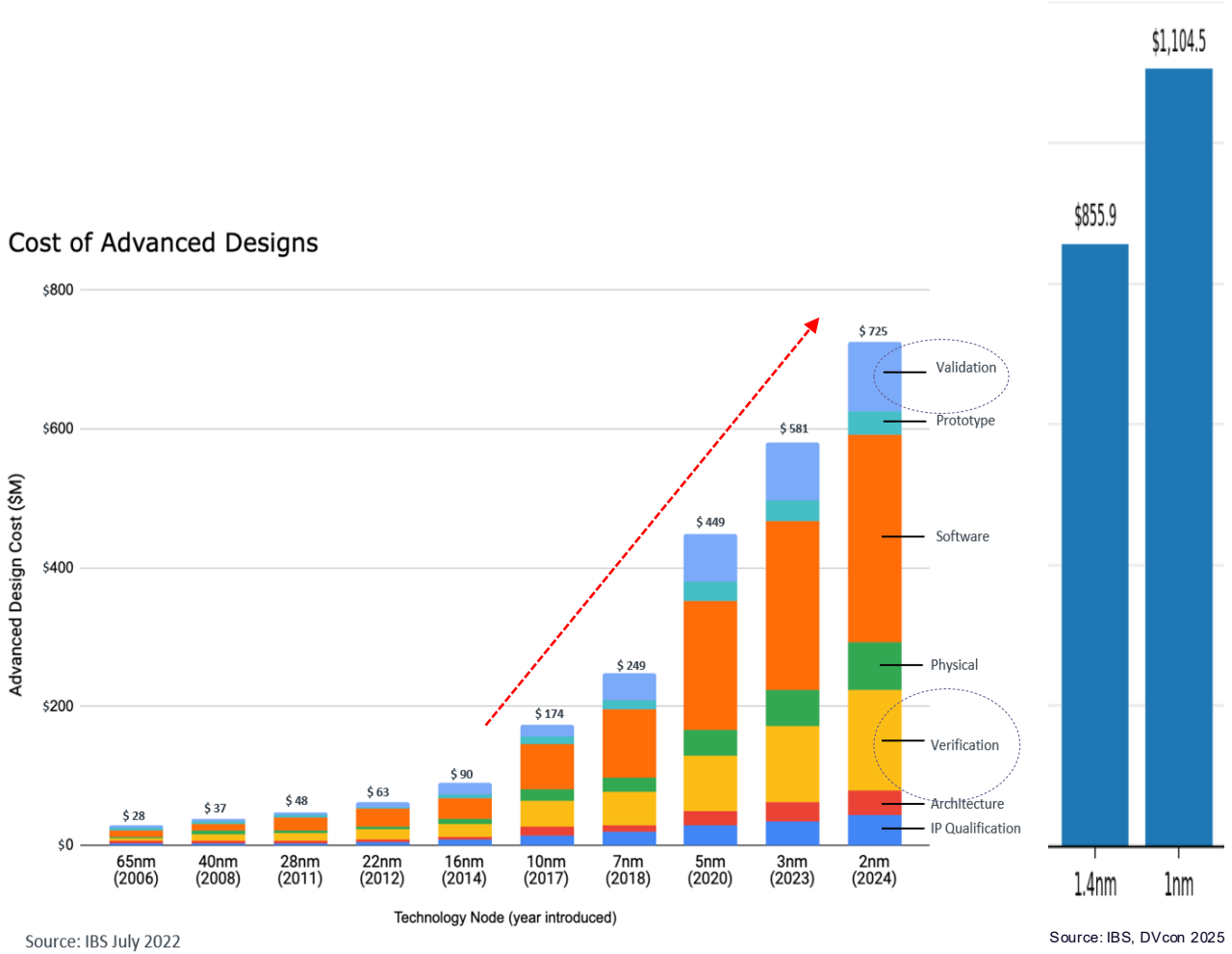
2024 Wilson Research Group IC/ASIC functional verification trend report

© 2026 Arm

ASIC industry trend - Design cost

- Overall cost and schedule of new chip designs is growing exponentially

- Verification/validation cost increased with
  - Emerging **technology** as the design **complexity** grows
  - Emerging **new layers** of design requirements
    - Safety, Security and hardware-software interactions in embedded processor designs



© 2026 Arm

What Verification actually involves?

It’s not a single test – it’s an engineering system that builds evidence and reduces silicon risk

A — PLAN
Requirements • risk • verification strategy • test/coverage plan

B — BUILD
Testbench • UVM agents • sequences • models • scoreboards • assertions

C — EXECUTE
Simulation • formal/static • emulation/FPGA • regressions • CI

D — MEASURE
Functional/code/assertion coverage • requirements traceability • metrics

E — DEBUG & CLOSE
Failure triage • root cause • bug fixes • coverage closure • waivers

F — SIGN-OFF
Evidence review • residual risk • open-bug status • readiness decision

PEOPLE

Verification engineers • designers • architects • SW/firmware • methodology/tool experts

INFRASTRUCTURE

EDA licences • compute farms • regression/CI • databases • dashboards • automation

Verification effort = people + methodology + reusable environments + EDA tools + compute + data + disciplined closure

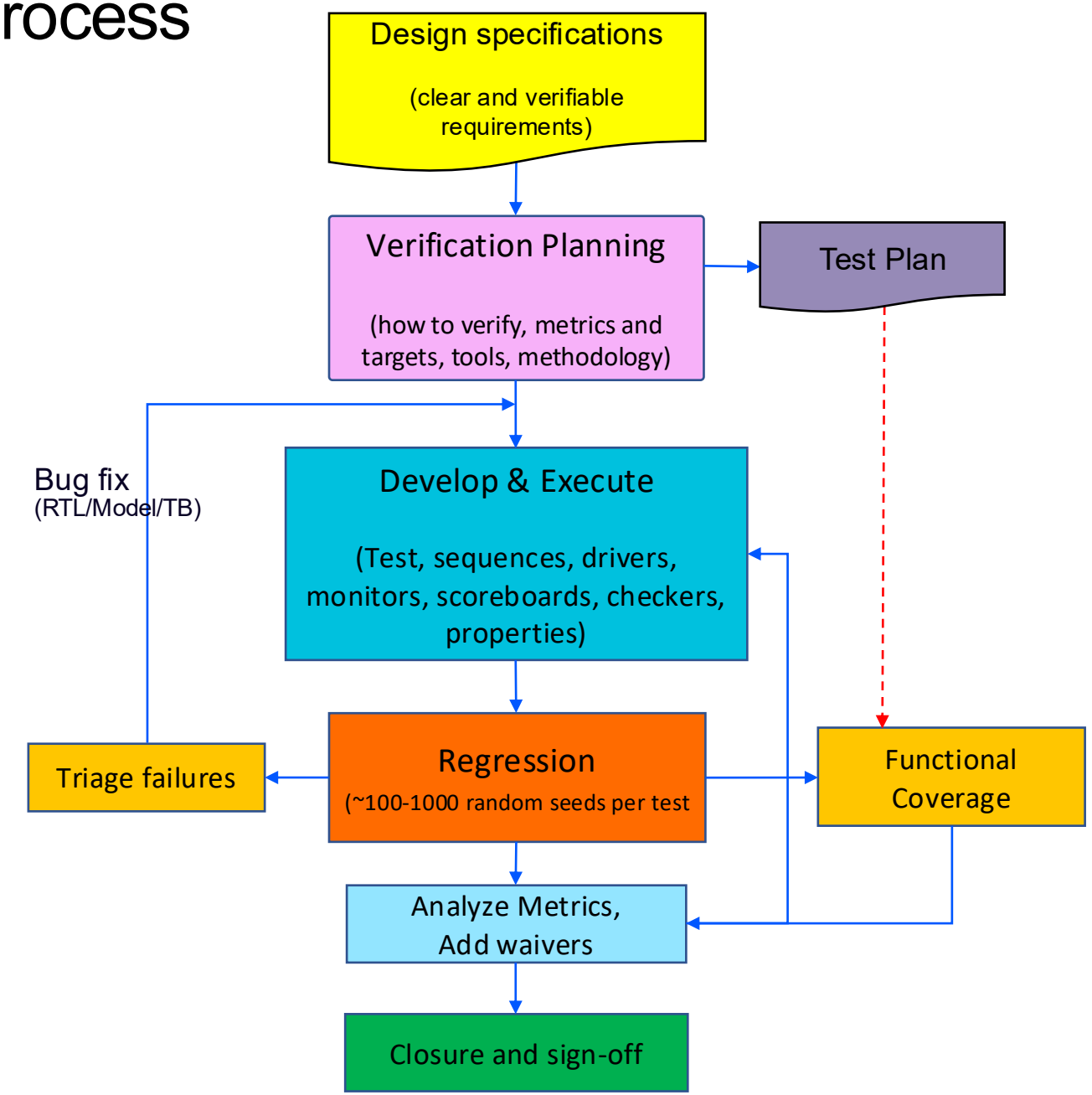
arm

© 2026 Arm

Verification process

© 2026 Arm

Verification Process



RTL and verification environment develop in parallel and converge continuously

Key principles

- Collaborative
- Metric-driven
- Iterative
- Evidence-based sign-off

© 2026 Arm

Verification techniques

© 2026 Arm

Design States and State Space

Why exhaustive simulation becomes impossible as hardware complexity grows

arm

1 What is a design state?

A snapshot of the values stored in the design at a particular time.

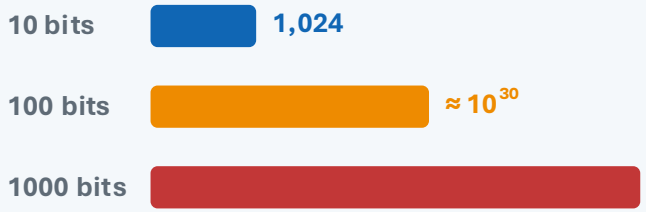
State is primarily defined by storage elements:

- **Flip-flops / registers**
- **Memory bits**
- **State variables**

Next state = f (current state, inputs)

For a synchronous design, the state changes on active clock edges according to the current state and inputs.

2 State-space explosion



Exhaustive simulation becomes impractical.

3 Verification implication

- ✓ We cannot simply test every possible state.
- ✓ Stimulus must target meaningful and rare reachable states.
- ✓ Coverage tells us which planned behaviors we have exercised.

Verification challenge: we cannot simply simulate every state — we need coverage-driven simulation, formal methods and abstraction.

© 2026 Arm

Major functional verification techniques

STATIC

Static Analysis

Lint / coding rules

CDC & RDC

Structural / connectivity checks

Best for

Fast structural checks without simulation

FORMAL

Formal Verification

Property checking

Equivalence checking

Formal applications

Best for

Proving targeted behavior and corner cases

DYNAMIC

Simulation

Directed tests

Constrained-random / UVM

Assertions, scoreboards & coverage

Best for

Rich functional scenarios and debug visibility

HARDWARE-ASSISTED

Emulation + FPGA

Long workloads / large SoCs

HW/SW co-verification

Software bring-up & interfaces

Best for

Speed, scale and system-level workloads

HOW THEY FIT

Static: structure **Formal: prove** **Simulation: behavior** **HW-assisted: scale**

No single technique finds all bugs — **combine** complementary evidence for sign-off.
Dynamic simulation remains to be the main method!

arm

© 2026 Arm

ASIC verification technology adoption trend

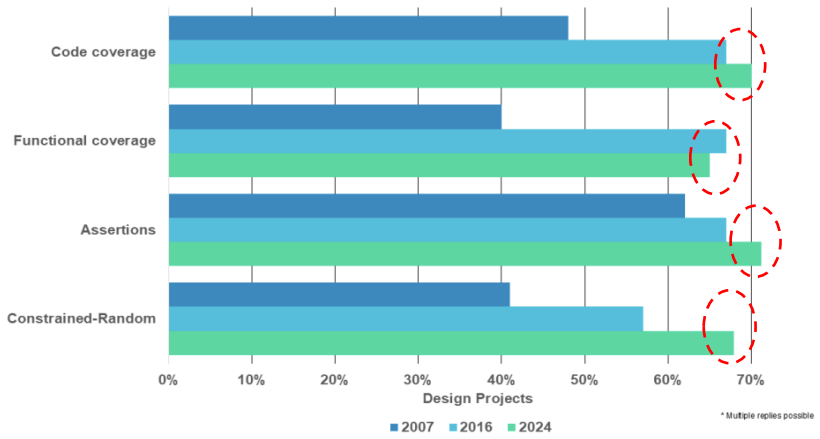


Figure 19. IC/ASIC project simulation technique trends.

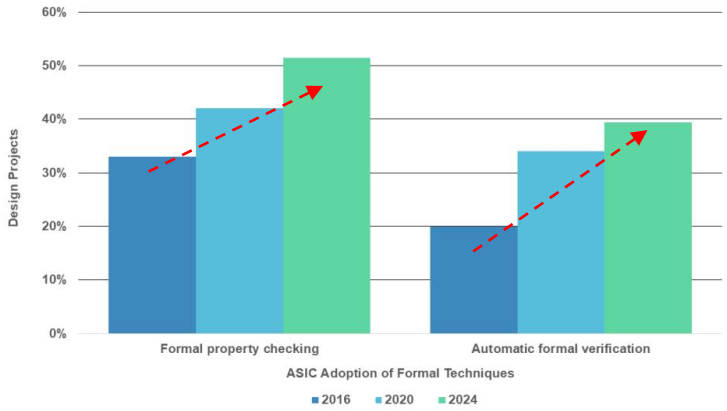


Figure 18. IC/ASIC project formal technology adoption trends.

- Overall **dynamic** verification technique adoption trends have remained flat

- **Static** verification techniques - **formal** property checking and automatic formal applications are growing



- Intelligent test generation
- Coverage closure
- Failure triage & debug
- Code/assertion creation
- Insight & prediction

To achieve higher quality, lower cost, and faster time to silicon – combination of simulation, formal, HW-assisted acceleration and AI-assisted verification are growing.

© 2026 Arm

Verification languages and methodology adoption trend

- Continual interest in **SystemVerilog** for verification.
- Accellera **UVM** is currently the predominant standard that has been adopted to create IC/ASIC testbenches.
- SystemVerilog Assertions (**SVA**) is the predominant assertion language in use today

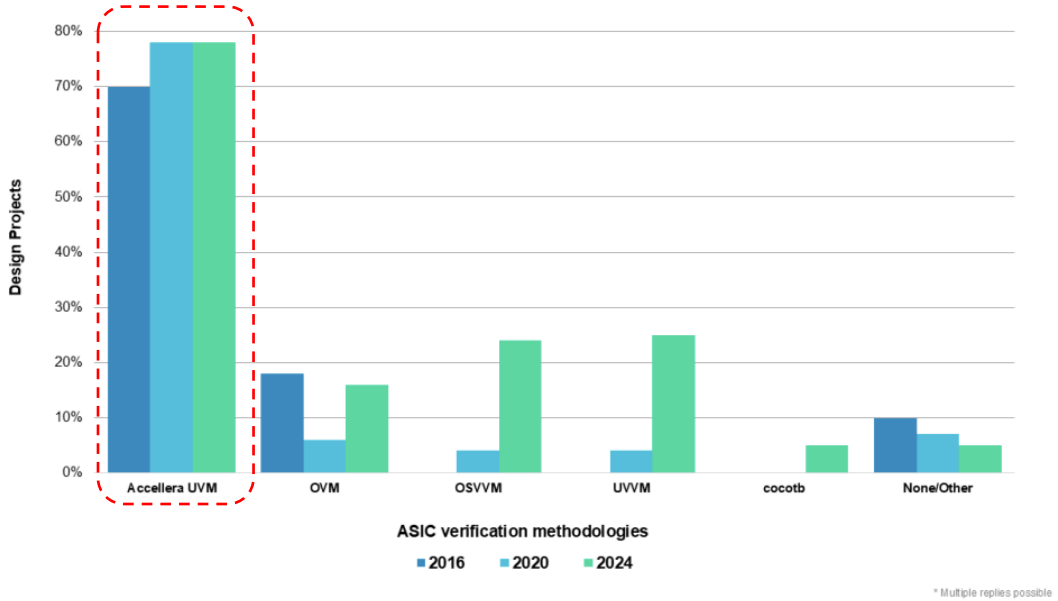


Figure 16. IC/ASIC project methodology and base-class libraries adoption.

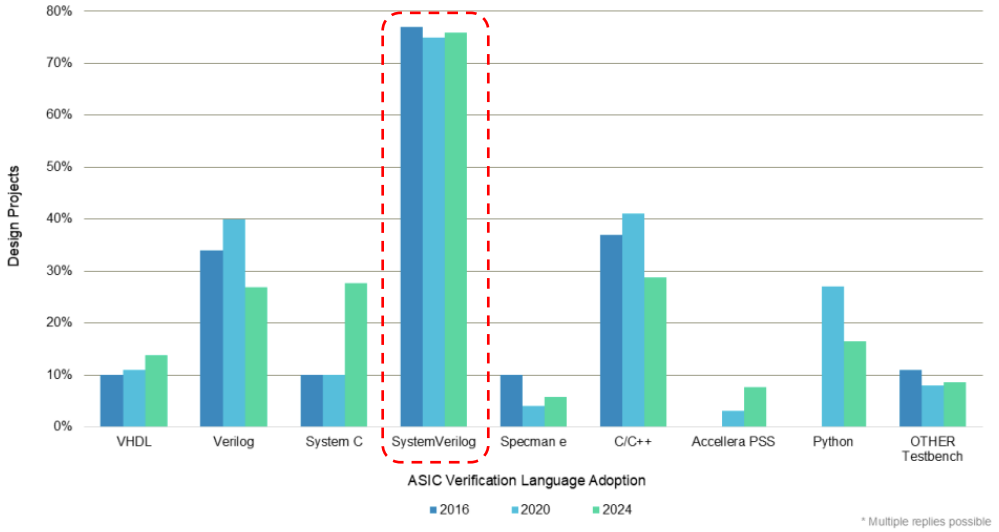


Figure 15. IC/ASIC project verification language adoption.

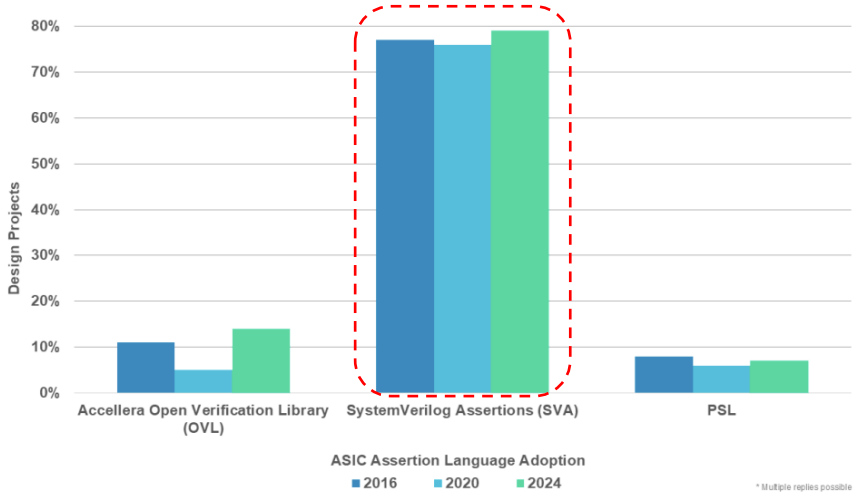


Figure 17. IC/ASIC project assertion language adoption.

© 2026 Arm

Static checks – catch structural bugs early

EDA-style static sign-off flow

RTL / Netlist
Design structure

Constraints
Clocks • resets • SDC/UPF

Static Engine
Lint • CDC • RDC

Findings
Violations • warnings

Fix • justify • waive

LINT / RTL QUALITY

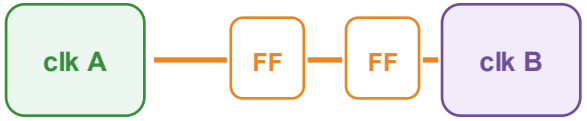
Find coding and structural issues



Unintended latches, Width/sign mismatches, Dead/unreachable logic, Suspicious constants

CDC

Check asynchronous clock crossings



Missing/incorrect synchronizers, Reconvergence hazards, Handshake assumptions, Data-loss/coherency risks

RDC

Check asynchronous reset crossings



Reset release hazards, Reset reconvergence, Metastability exposure, Power/reset interactions

© 2026 Arm

Formal Verification – mathematically prove correctness

Formal verification flow

RTL Model
Design under verification

Properties
Intended behavior
(assertions/assumptions)

Formal Tool
Model checker

check specified properties hold for all behaviors allowed by the model/assumptions

Proved OK
All properties hold

Counter Example (simulate)
Reproduce & debug

A failed property produces a CEX that can be replayed

KEY ADVANTAGES

No conventional testbench or stimulus setup required

Finds deep corner cases that simulation may miss

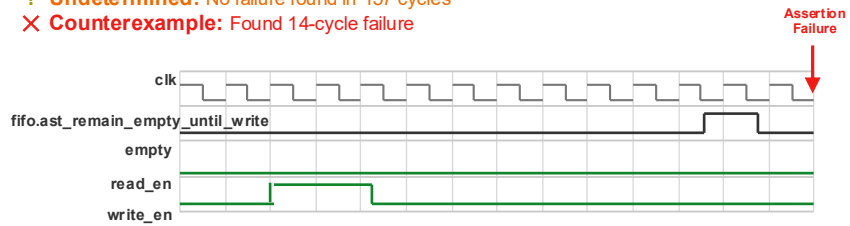
Excellent for control logic, protocols, safety/security properties

WHAT IT DOES

Mathematically check properties

	Type	Name	Engine	Bound
●	Assume	fifo.asm_no_write_when_full	?	
●	Assume	fifo.asm_no_read_when_empty	?	
✓	Assert	fifo.ast_no_full_and_empty	N (3)	Infinite
?	Assert	fifo.ast_remain_full_until_read	B	157 -
✗	Assert	fifo.ast_remain_empty_until_write	N	14

✓ **Full Proof:** Impossible to violate this assertion
? **Undetermined:** No failure found in 157 cycles
✗ **Counterexample:** Found 14-cycle failure



LIMITATIONS

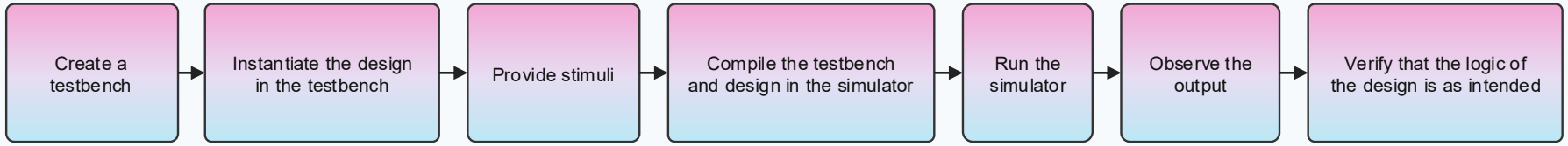
Proof complexity grows rapidly on large/deep designs

Good properties / assumptions require engineering skill

© 2026 Arm

Functional Simulation – verify behavior through simulation

Functional simulation flow

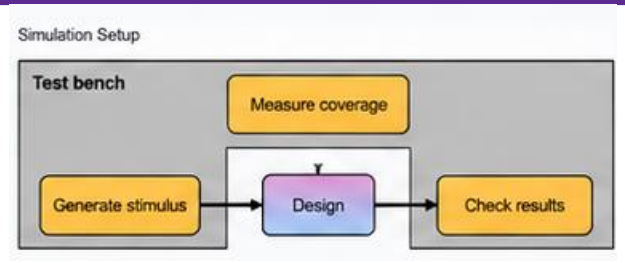


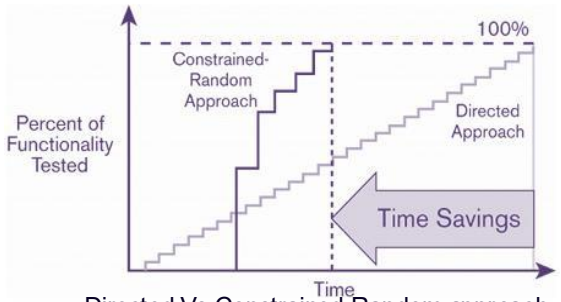
KEY ADVANTAGES

- Highest controllability and debug-visibility in the design
- Reproducible test scenarios
- Functional bugs are identified early in design

HOW IT WORKS

- DUT is simulated over time through simulation software
- A testbench is developed,
  - To apply stimulus on inputs (directed / constrained-random)
  - Automatically check behavior
  - Measure what was exercised
- Results – waveforms, logs, coverage
- Constrained random verification reaches coverage target earlier





Directed Vs Constrained Random approach

LIMITATIONS

- Explores only the scenarios actually simulated
- Takes large amount of computing resources

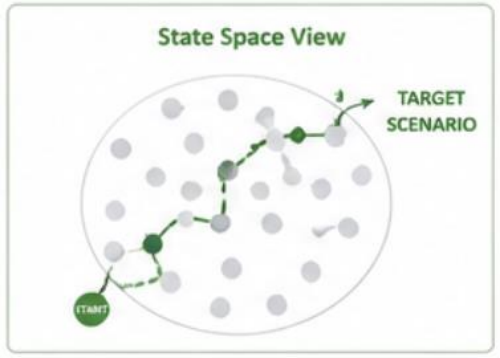
© 2026 Arm

Exploring large state space

Directed simulation, constrained-random simulation and formal solve different verification problems.

1 Directed Simulation

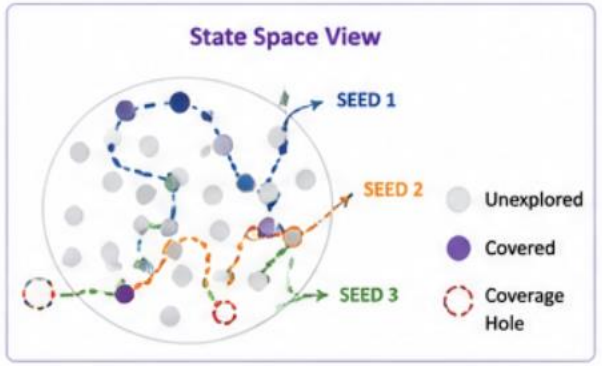
CONTROL



- Engineer specifies the scenario and stimulus
- Coverage is narrow unless many tests are written
- Bring-up, checking specific features, corner cases, bug reproduction, and directed regressions.

2 Constrained-Random

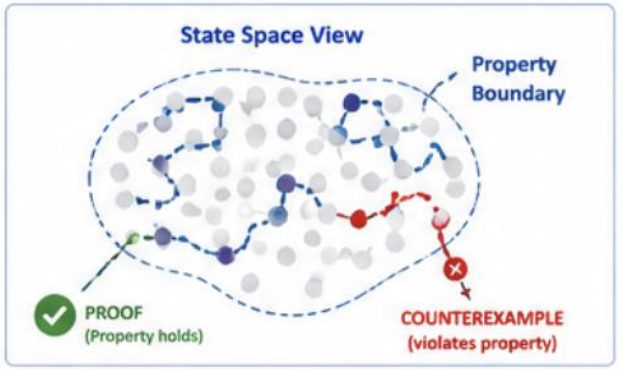
BREADTH



- Constraints generate many legal stimulus combinations
- Coverage guides regressions and exposes unanticipated interactions
- Broad exploration — but no guarantee that every relevant state is reached
- Complex protocols, large SoCs, and discovering unexpected bugs across broad scenario space.

3 Formal Verification

PROOF



- Properties define the behavior that must always hold
- Tool mathematically explores reachable behaviors under assumptions
- Can prove a property or return a counterexample; capacity depends on complexity
- Protocol correctness, safety properties, corner cases, and bug hunting at depth

Best practice: Use directed tests for intent, constrained-random for breadth, and formal for properties that need proof.

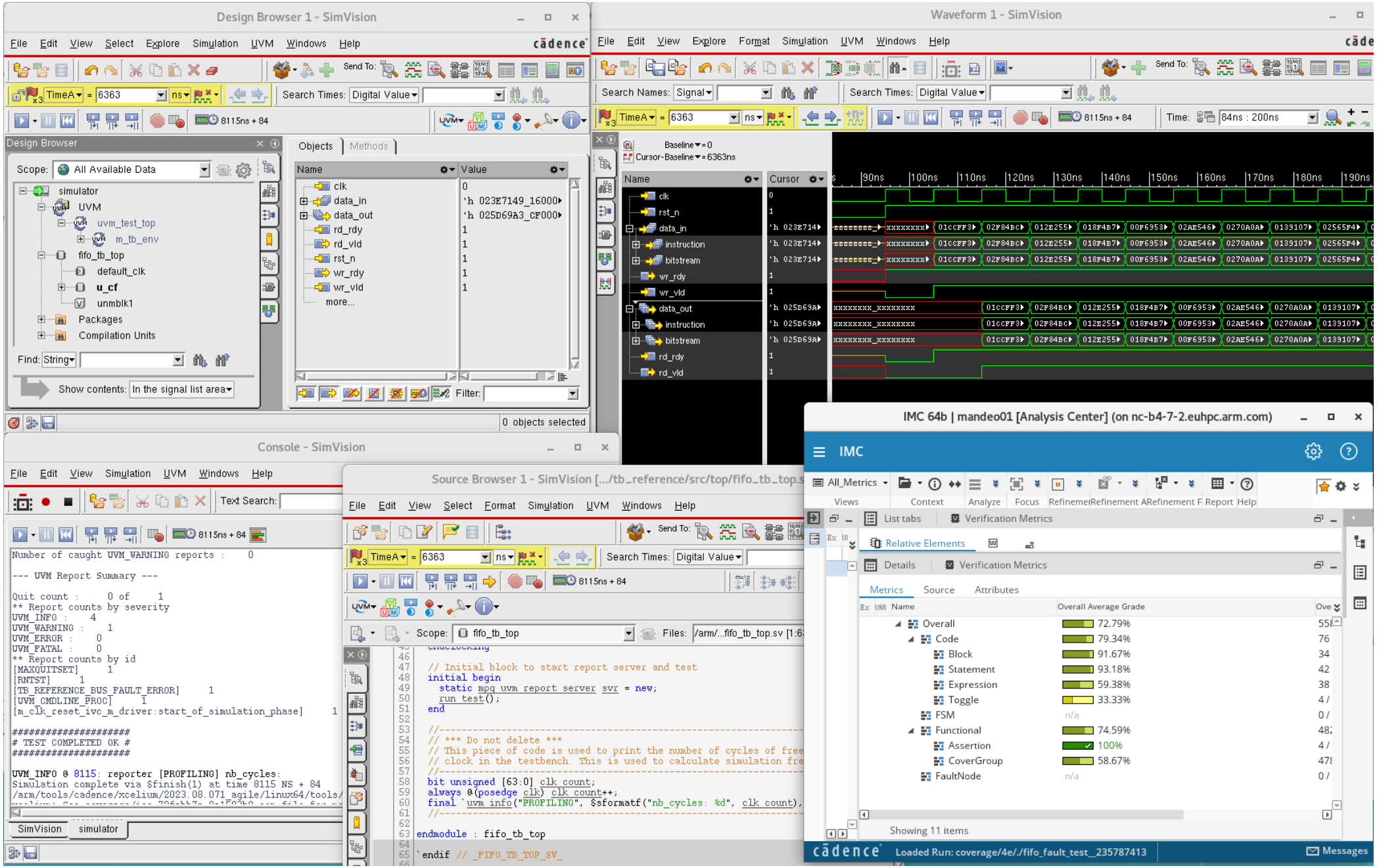
COMBINE THEM

arm

© 2026 Arm

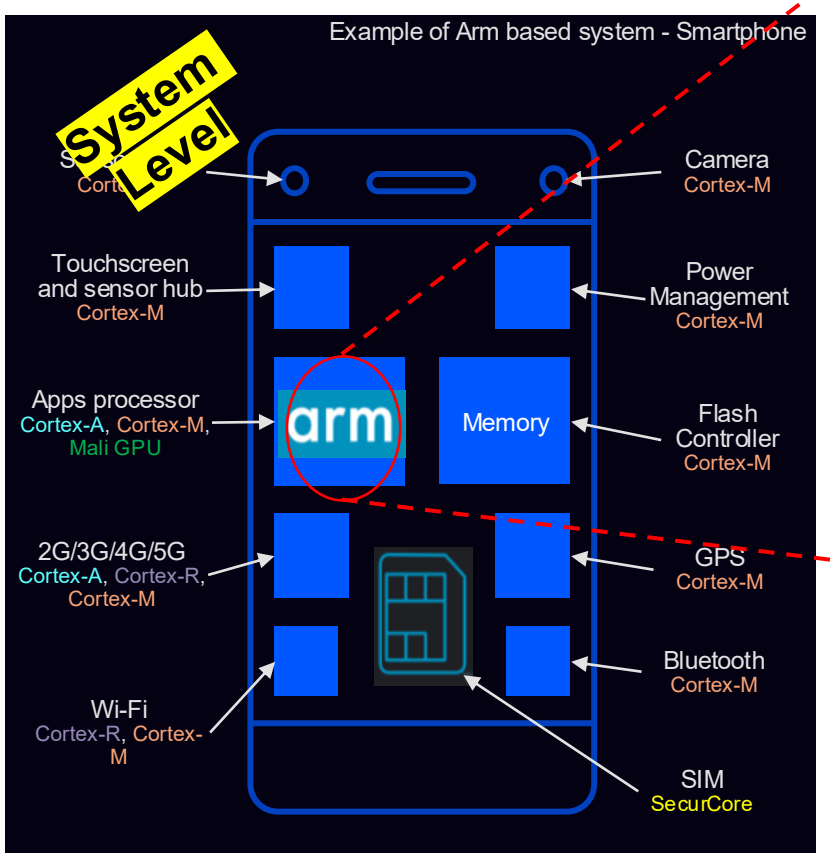
Simulator

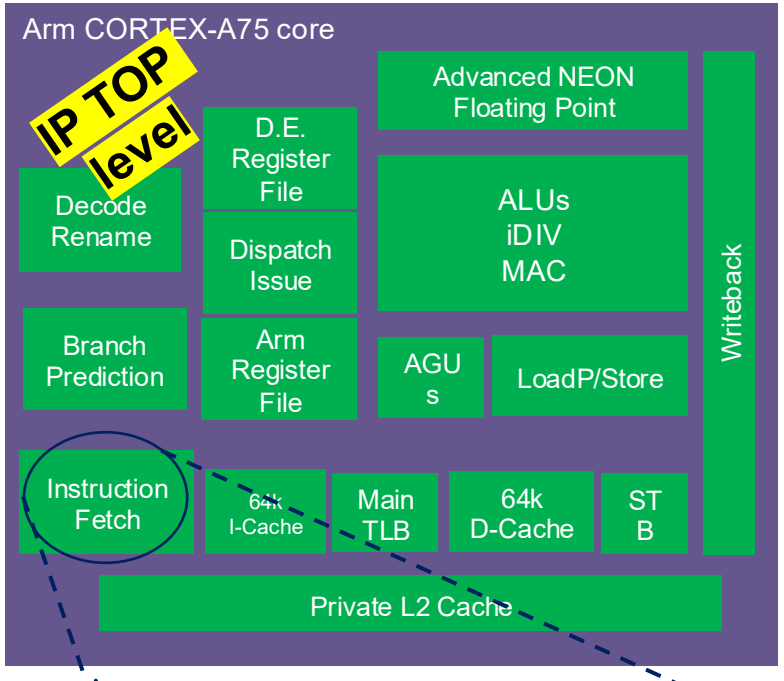
- Design browser
- Waveform window
- Console output
- Source browser
- Coverage view

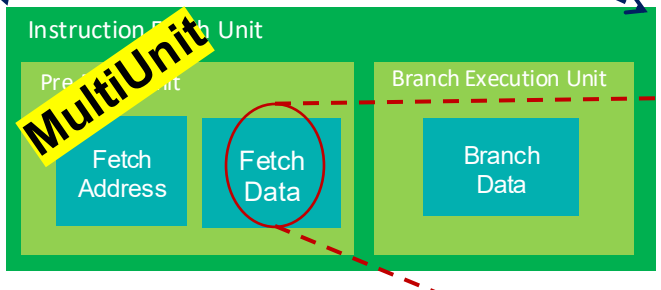


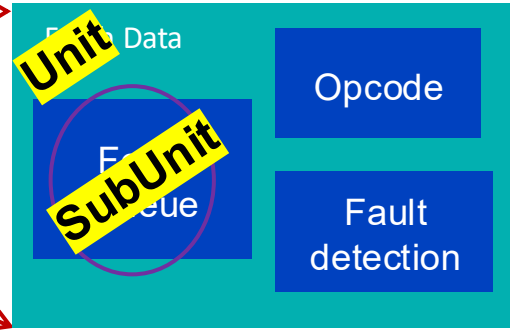
© 2026 Arm

Design complexity levels









- Verification approaches
  - Bottom-up
  - Top-down

© 2026 Arm

Verification focus at different abstraction levels

Sub-Unit/ Unit

Deep functional correctness

- Verify **detailed functionality**, corner cases and local interfaces
- Check protocols, data integrity and control behavior
- **Target** 100% code and functional coverage of unit features and interfaces
- Use simulation and formal

Multi-Unit/ IP-Top

Integration correctness

- Verify **connectivity** and protocols between units
- Exercise top-level configuration and architectural use-cases
- **Target** functional coverage related to integration critical features and interfaces
- Use simulation and focused formal

System (SoC)

End-to-end behavior

- Verify IP/memory connectivity and **end-to-end data paths**
- Exercise representative HW/SW and system use-cases, high-value scenarios
- **Target** all use cases pass
- Use Emulation / FPGA prototyping for long, SW-driven workloads

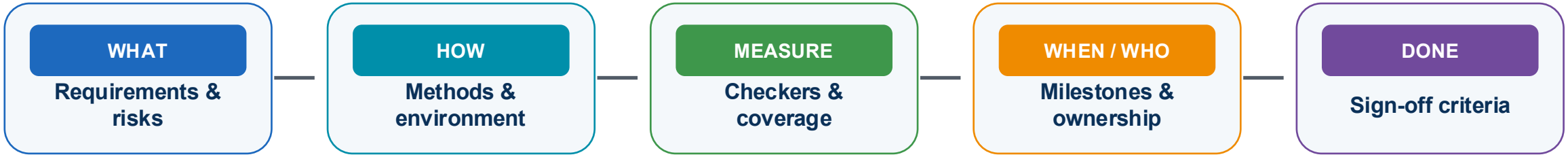
© 2026 Arm

Verification Planning

© 2026 Arm

Verification planning

Turn requirements into an executable, measurable verification evidence



A good vPlan / test plan should contain

- Trace each verifiable requirement to a method and evidence
- Define stimulus, checkers/assertions and functional coverage
- Identify tools, models, environments and dependencies
- Prioritize by risk; assign owners and milestones
- Define objective closure and waiver criteria

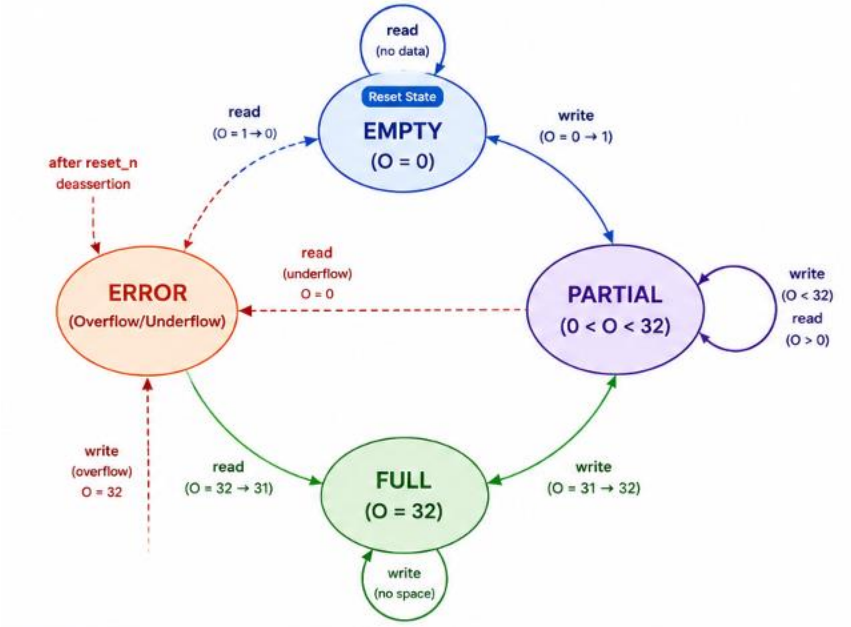
Coverage

- **Code coverage** tells us what RTL was exercised
- **Functional coverage** tells us whether planned behaviors and scenarios were observed.

© 2026 Arm

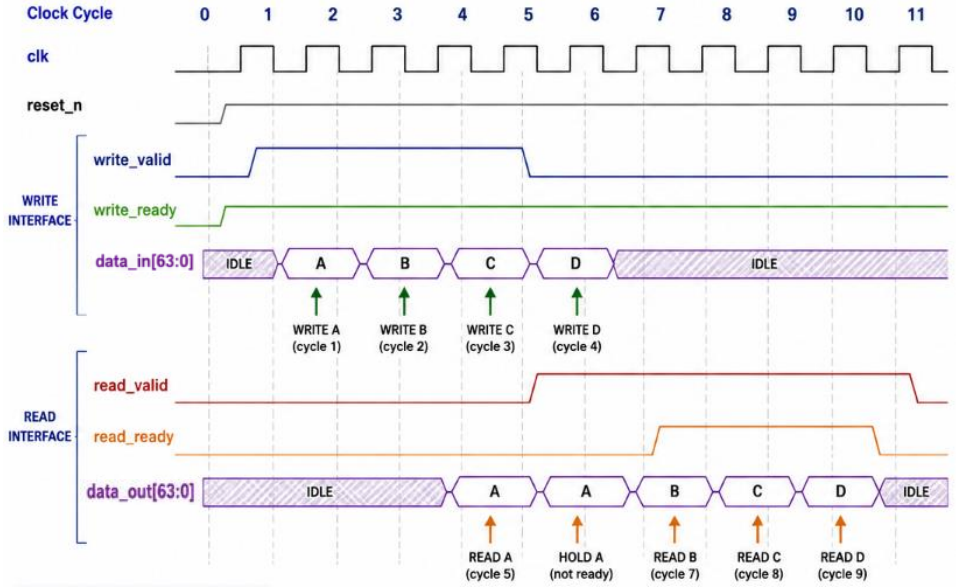
Example Design – FIFO





Specifications on data

Data [63:0]	Field	Valid values
63:56	opcode	00 = NOP 01 = ADD 02 = SUB
55:40	operand_a	[0000:FFFF]
39:24	operand_b	[0000:FFFF]
23:0	dest	00 = REG 01 = SRAM 02 = DRAM



© 2026 Arm

FIFO Example Verification strategy

Turn requirements into checks, coverage and measurable sign-off evidence.

1 Choose methods

Simulation / UVM for behavior & scenarios
Complementary methods – Formal/Static checks

2 Test environment requirements

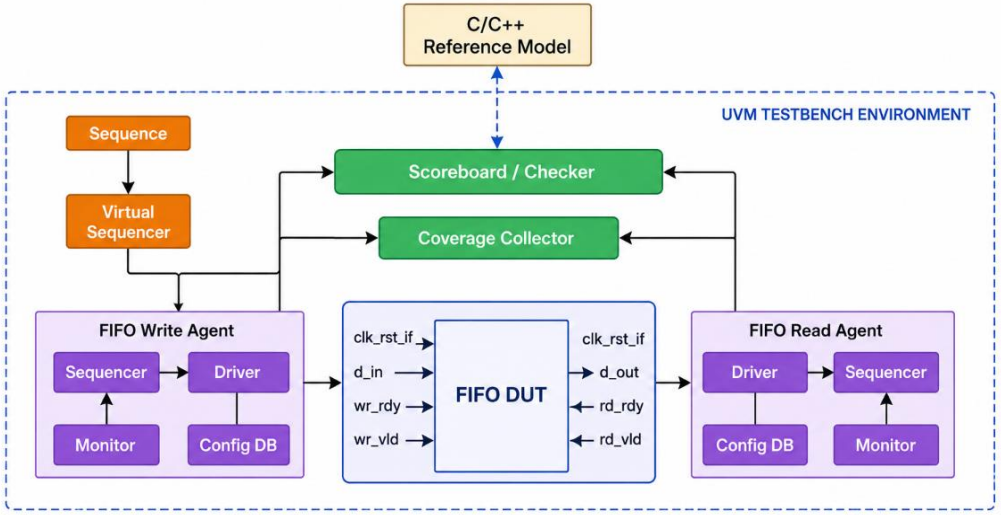
Write, read clk/rst agents • monitors • scoreboard
Reference model • coverage collector

3 Target high-value scenarios

Ordering & no loss • valid/ready protocol • reset
Empty/full • wrap-around • backpressure • illegal payload

4 Define sign-off criteria

Requirements verified • regressions stable
Coverage targets met or justified • no unacceptable open bugs



Code coverage

(structural)

- Statement / branch
- Condition / expression
- Toggle (where useful)
- FSM state / transition

Functional coverage

- Occupancy: empty / partial / full
- Read/write handshake outcomes
- Backpressure duration
- Reset during activity
- Cross coverage - Opcode × destination

Assertions / checks

- No overflow / underflow
- Ordering preserved
- Handshake protocol rules
- Reset behavior
- Data integrity / no duplication

© 2026 Arm

FIFO Testplan

REQ ID	REQ details	Test scenario or stimulus	Details of checker or coverage	Notes
ID_0001	**Reset**	Apply rst_n low at t0 with idle interfaces	After deassert, FIFO empty; rd_vld==0; wr_rdy==1	Async negedge; release async then 2 cycles settle
ID_0002	**Mid-sim reset**	Assert rst_n low when wr_vld=1 and/or rd_vld=1	No handshakes during reset; pointers/state cleared; no stale rd_data after release	Try multiple relative phases
ID_0003	**FIFO empty**	rd_rdy=1 while FIFO empty	rd_vld==0; no pops in model	
ID_0004	**FIFO full**	Fill to full, hold wr_vld=1	wr_rdy deasserts within 1 cycle; no overflows	Requires depth knowledge
ID_0005	**Single write/read**	Push 1, then pop 1	Data matches; order preserved	Smoke
ID_0006	**Burst write only**	Push N=depth, no reads	wr_rdy drops at full; scoreboard count==depth	
ID_0007	**Burst read only**	Preload N, then read out	All data/order correct; empty toggles	
ID_0008	**Simultaneous R/W**	Continuous wr_vld=1, rd_rdy=1	No stalls after pipeline fill; data order correct	Stress
ID_0009	**Backpressure wr**	Randomly toggle wr_vld with DUT backpressure	No lost/dup entries; handshake count==model count	Constrained-random
ID_0010	**Backpressure rd**	Randomly toggle rd_rdy	Data stable while rd_vld=1 && rd_rdy=0	SVA checks
ID_0011	**Boundary crossings**	Oscillate around empty↔1 and full↔full-1	Correct deassert/assert of rd_vld/wr_rdy	
ID_0012	**Pointer wrap**	Push > depth then pop > depth multiple times	No corruption across wrap	Long run
ID_0013	**Payload legality**	Generate all legal opcodes × destinations, plus illegal values	Legal values pass through; illegal occurrences counted/flagged	DUT may not check; we still cover
ID_0014	**NOPs in stream**	Intermix NOP with ADD/SUB	Preserves order and content	
ID_0015	**min/max operands**	Sweep operand_a/b extremes (0x0000…/0xFFFF…)	Payload unchanged	
ID_0016	**Latency throughput**	Measure cycles from push to first vld pop under no stalls	Meets spec (if any); otherwise record	
ID_0017	**Quiescent stability**	Hold all inputs steady; no handshakes	Outputs stable; no glitches	

© 2026 Arm

FIFO Coverage plan

Coverage legend (bin names → intent)

CP_OPC: opcode {NOP, ADD, SUB, illegal}

CP_DEST: destination {REG, SRAM, DRAM, illegal}

CP_A / CP_B: operand_a / operand_b {zero, middle, max} (middle is a representative range)

CR_OPC×DEST: cross of opcode × destination (legal triples must reach 100%)

CP_OCC: occupancy {0,1,[2:depth-2],depth-1,depth} (empty/low/mid/high/full)

EV_BOUND: boundary events {empty→nonempty, nonempty→empty, full→nonfull, nonfull→full}

CP_WH: write handshake state {H=wr_valid&wr_ready, S=wr_valid&!wr_ready, I=!wr_valid&wr_ready

CP_RH: read handshake state {H=rd_valid&rd_ready, S=rd_valid&!rd_ready, I=!rd_valid&rd_ready}

CP_STALL: wait cycles before handshake {1, [2:3], [4:7], 8+}

CP_BURST_W/R: consecutive write/read handshakes {1, [2:3], [4:7], 8+}

CP_SIMRW: at least one cycle with simultaneous write & read handshakes

CP_WR_STABLE / CP_RD_STABLE: sampled while valid=1 && ready=0 to confirm data stability

CP_RST_PHASE: reset timing {idle, wr-wait, rd-wait, simRW}

CP_RST_WIDTH: reset length {1, [2:3], 8+}

CP_WRAP: pointer wrap sampled (push or pop crosses depth boundary)

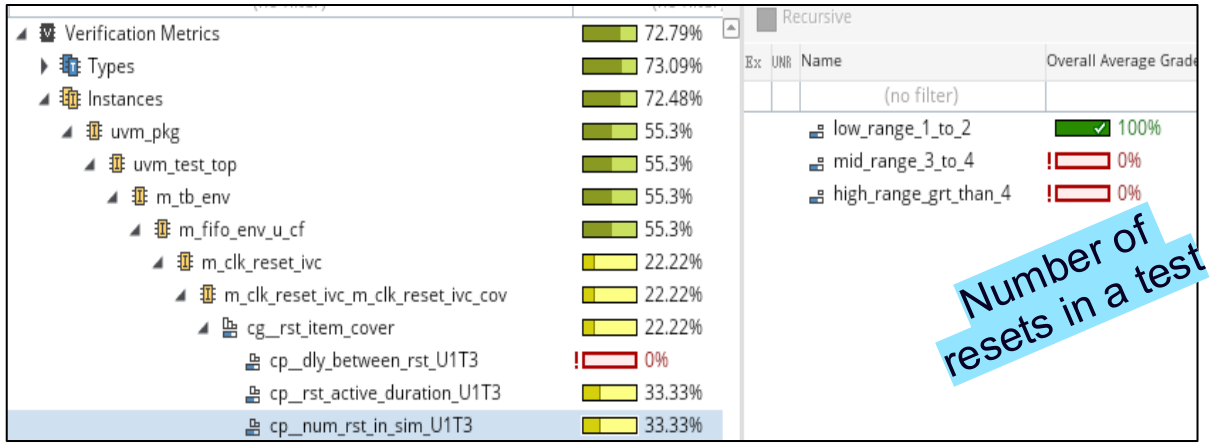
REQ ID	REQ details	Coverage (items • bins/ranges targeted)
ID_0001	**Reset**	**CP_RST_PHASE**•idle; **CP_RST_WIDTH**•[2:3]; **CP_OCC**•0; **CP_RH**•I; **CP_WH**•I; **EV_BOUND**•nonempty→empty (post-reset steady)
ID_0002	**Mid-sim reset**	**CP_RST_PHASE**•wr-wait, rd-wait, simRW; **CP_RST_WIDTH**•1 and 8+; **CP_WR_STABLE/CP_RD_STABLE** sampled before reset; **EV_BOUND**•→empty on
ID_0003	**FIFO empty**	**CP_OCC**•0; **CP_RH**•I; ensure **CP_STALL** not sampled (no valid)
ID_0004	**FIFO full**	**CP_OCC**•depth; **EV_BOUND**•nonfull→full; **CP_WH**•S (backpressured);
ID_0005	**Single write/read**	**CP_BURST_W**•1; **CP_BURST_R**•1; **CP_OCC**•0→1→0; **EV_BOUND**•empty→nonempty,
ID_0006	**Burst write only**	**CP_BURST_W**•8+ (as needed); **CP_OCC** sweeps 0..depth; **EV_BOUND**•nonfull→full;
ID_0007	**Burst read only**	**CP_BURST_R**•8+; **CP_OCC** sweeps depth..0; **EV_BOUND**•nonempty→empty; **CP_RH**•H
ID_0008	**Simultaneous R/W**	**CP_SIMRW**•hit; **CP_BURST_W/R**•8+; **CP_OCC** hover mid; **CP_WH/RH**•H concurrent
ID_0009	**Backpressure wr**	**CP_WH**•H,S,I all; **CP_STALL**•1,[2:3],[4:7],8+ (write side); **CP_WR_STABLE** sampled in S; **CP_OCC** crosses high/full
ID_0010	**Backpressure rd**	**CP_RH**•H,S,I; **CP_STALL** (read side)•1,[2:3],[4:7],8+; **CP_RD_STABLE** sampled in S;
ID_0011	**Boundary crossings**	**EV_BOUND**•all four events; **CP_OCC**•0,1,depth-1,depth; **CP_WH/RH** transitions across
ID_0012	**Pointer wrap**	**CP_WRAP**•hit (push & pop); **CP_OCC** sweep; **CP_BURST_W/R**•8+; **CR_OPC×DEST**
ID_0013	**Payload legality**	**CP_OPC**•NOP,ADD,SUB,illegal; **CP_DEST**•REG,SRAM,DRAM,illegal; **CR_OPC×DEST** all legal combos (100%); **CP_A/B** cover {zero,middle,max}
ID_0014	**NOPs in stream**	**CP_OPC** emphasize NOP; **CR_OPC×DEST** with NOP×{REG,SRAM,DRAM}; **CP_OCC** mid;
ID_0015	**min/max operands**	**CP_A/B**•zero,max heavy; include middle; cross with **CP_OPC** {ADD,SUB,NOP} and **CP_DEST** {REG,SRAM,DRAM} (sampled)
ID_0016	**Latency throughput**	**CP_SIMRW** optional; collect **CP_STALL** distribution (read path); **CP_OCC** from ramp-up; tag performance bins (custom histogram)
ID_0017	**Quiescent stability**	**CP_WH/RH**•I persistence; sample **CP_WR_STABLE/CP_RD_STABLE** not applicable (no

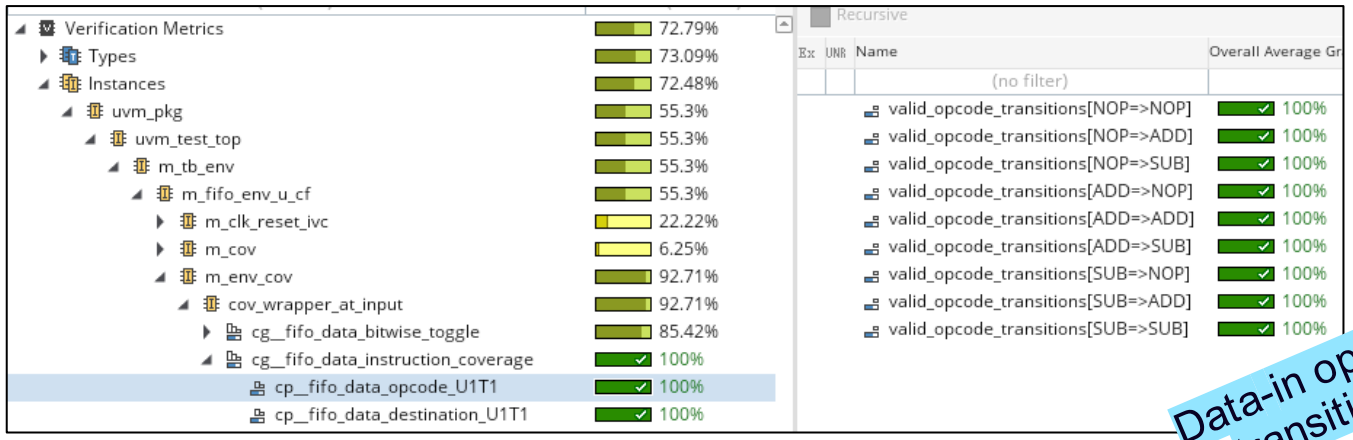
© 2026 Arm

FIFO example – Measure progress and close coverage

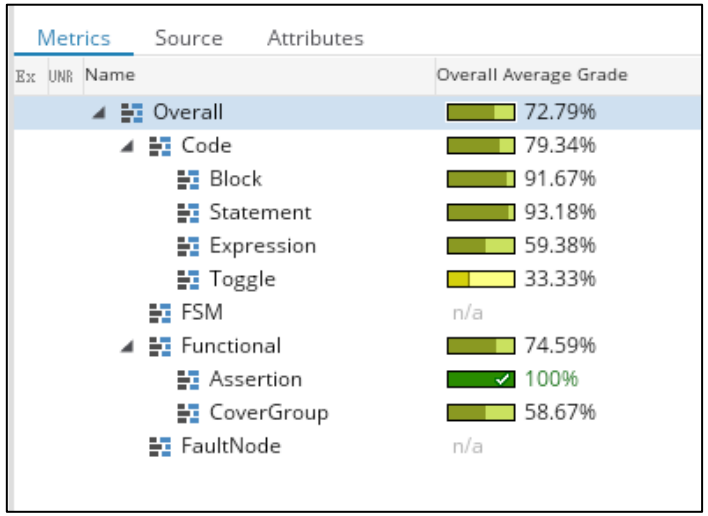
Coverage closure = analyze holes → classify reachable / unreachable / uninteresting → improve stimulus or checks → justify exclusions / waivers.

Fifo functional coverage





Fifo code coverage



© 2026 Arm

Verification Sign-off

© 2026 Arm

Regression testing



This Photo by Unknown Author is licensed under CC BY-NC-ND

Analyze data from regression, debug failures, analyze coverage holes

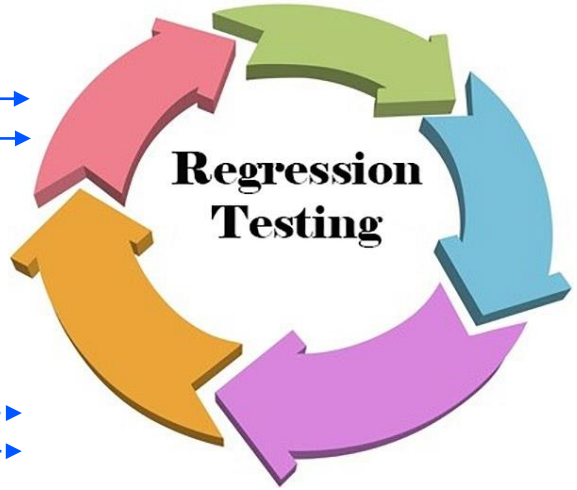
Change in code of Design or Verification files



Virtual user

Run few tests in sanity regressions to accept/reject the code

Run all tests, each with 1000s of seeds, in a nightly regression to ensure nothing else is broken



This Photo by Unknown Author is licensed under CC BY-NC-ND

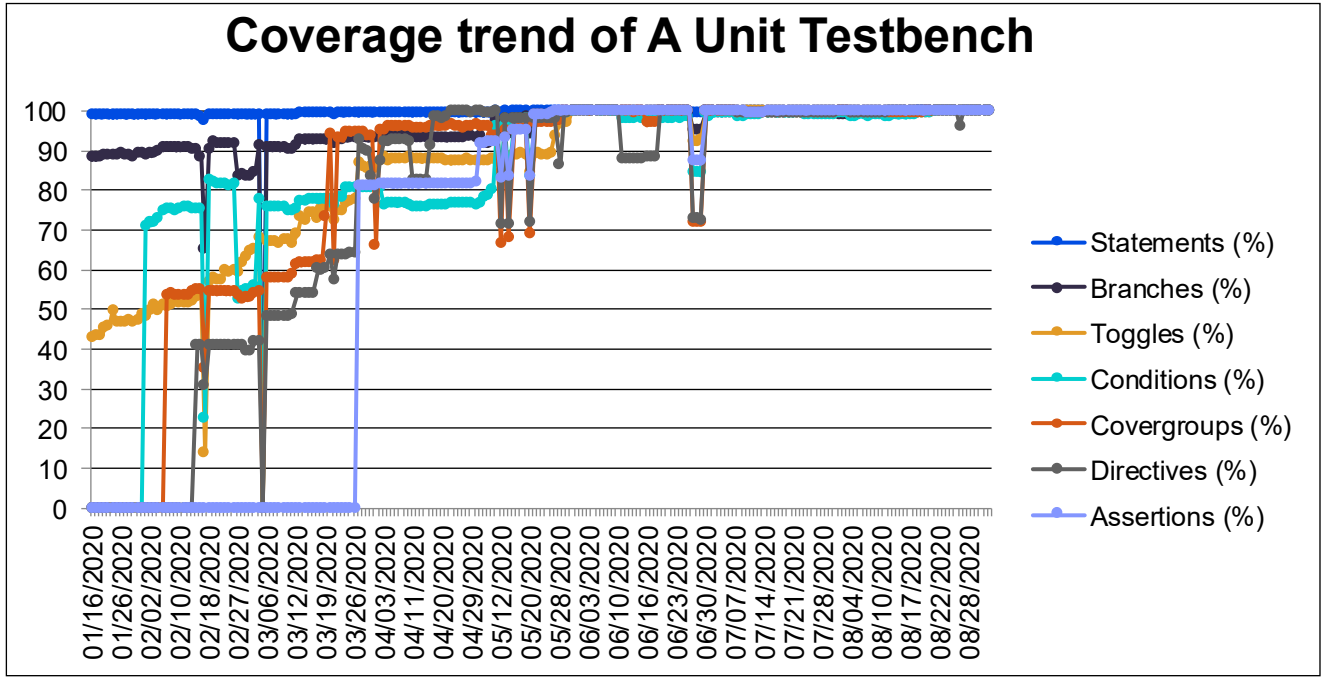


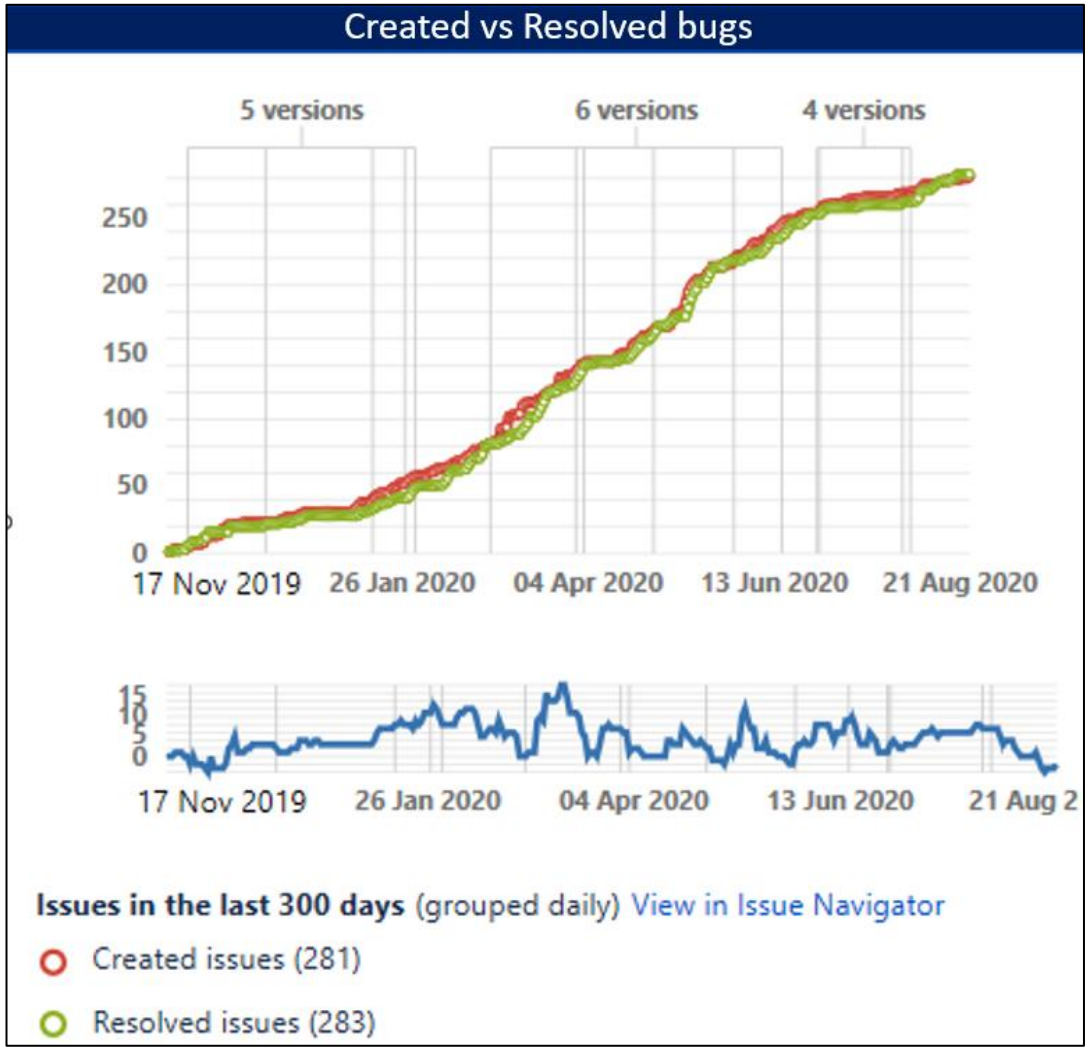
This Photo by Unknown Author is licensed under CC BY-NC

© 2026 Arm

Verification sign-off

- Verification is complete when,
  - All design requirements are verified
  - Coverage is stable after reaching the targets.
  - All critical bugs are fixed, and no new bugs for ~week/s
  - Justified exclusions/waivers and residual risk assessed





© 2026 Arm

AI in Verification

© 2026 Arm

Key challenges in verification and How AI helps

State-space growth, data volume, compute cost and engineering bandwidth

STATE SPACE

Huge combinations of states, configurations and corner cases

AI prioritizes promising tests, constraints and unexplored coverage

COVERAGE CLOSURE

Coverage holes become harder and more expensive to close

ML learns coverage/stimulus relationships and guides the next runs

REGRESSION SCALE

Thousands of tests, seeds and configurations consume compute

AI ranks, selects and de-duplicates regressions

DEBUG DATA

Logs, waveforms and failures overwhelm manual analysis

AI clusters failures and ranks suspicious signals / code / changes

KNOWLEDGE COMPLEXITY

Large specs, UVM environments and legacy flows slow ramp-up

GenAI assists search, explanation, requirements and artifact drafting

SYSTEM COMPLEXITY

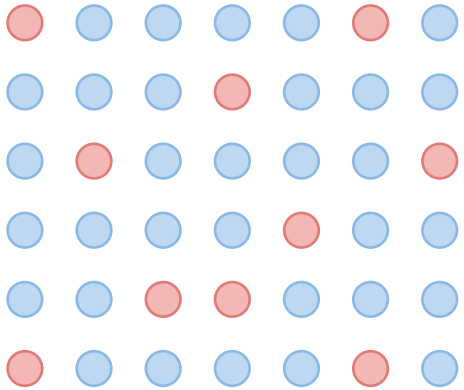
SoCs combine IP, software, power, clocks, security and chiplets

Agentic workflows coordinate multiple verification engines and data

© 2026 Arm

State-space explosion and coverage closure

Traditional closure



Coverage holes

Manual loop

- Run regressions
- ↓
- Merge coverage
- ↓
- Inspect holes
- ↓
- Tune tests / constraints
- ↓
- Run again

AI-assisted closure

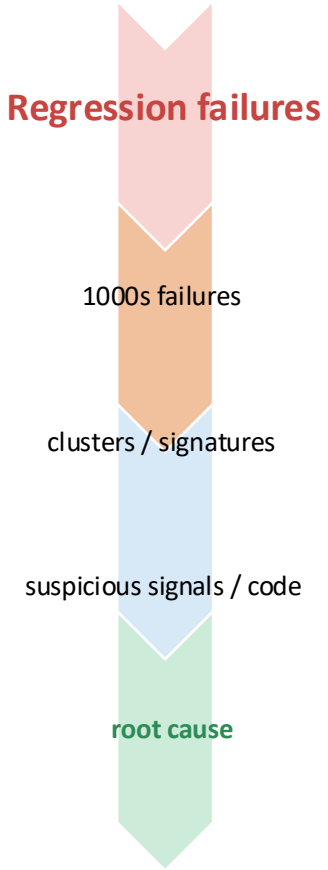
- Coverage + RTL + stimulus
- ↓
- Root-cause analysis of unhit bins
- ↓
- Rank tests / knobs / constraints
- ↓
- Target next runs
- ↓
- Learn from new results

Modern EDA tools use ML to identify regression redundancy, analyze why coverage bins are not hit, and guide tests/constraints toward coverage targets.

AI changes coverage closure from “run more” toward “run the most informative next experiment.”

© 2026 Arm

Debug - Too much Data, Too little time



A single RTL defect may create thousands of failing tests and millions of waveform transitions.

D Useful Data sources

- Fail/Pass tests
- Logs and assertions
- Waveforms
- RTL revisions / code changes
- Coverage & configuration metadata

AI AI-ASSISTED DEBUG

- Cluster similar failures
- Compare passing vs failing runs
- Summarize SmartLogs / messages
- Rank suspicious signals and RTL
- Correlate failures with code changes
- Focus waveform inspection

© 2026 Arm

Verification knowledge, Planning and Creation

Specifications, protocols, UVM code and historical bugs are distributed across many documents and tools.

SPEC → PLAN

Extract candidate requirements, scenarios and traceability links

AI planning

PLAN → ARTIFACT

Draft tests, sequences, assertions, coverpoints and scripts

AI creation

ENV → EXPLAIN

Explain unfamiliar UVM components, protocols and legacy code

AI knowledge

RESULT → REVIEW

Summarize regressions, coverage holes and milestone status

AI review

Generated tests/properties/coverage are candidate artifacts — compile, lint, simulate/formally verify, review and regress them.

© 2026 Arm

How to use GenAI safely in Verification

Good GenAI tasks

Explain a failure
Draft boilerplate / assertions
Summarize logs/specs
Suggest coverage scenarios

Needs strong validation

Generated UVM/SVA
Root-cause hypotheses
Requirements extraction
Coverage-hole stimulus

Do not delegate blindly

Final sign-off
Waiver justification
Safety/security claims
“Bug-free” conclusions

AI finds patterns and optimizes workflows • Verification engines provide evidence • Engineers own intent, judgment and sign-off

© 2026 Arm

Summary

© 2026 Arm

Summary

Verification is not “running tests” — it is building measurable evidence that the implementation conforms to the specification.

WHY IT MATTERS

- **Find bugs early** — before tape-out
- **Reduce risk** — re-spins • schedule • product impact
- **Build confidence** — with evidence, not intuition



REMEMBER

Verify early	Combine methods	Close with evidence
Plan deliberately	Measure what matters	Use AI with engineering judgement

© 2026 Arm

Q & A



This Photo by Unknown Author is licensed under CC BY-SA-NC

© 2026 Arm

arm

Tack
ಧನ್ಯವಾದಗಳು
Merci
Danke
Gracias
Grazie
谢谢
ありがとう
Asante
Thank you
감사합니다
धन्यवाद
Kiitos
شكرًا
ধন্যবাদ
תודה
ధన్యవాదములు
Köszönöm

Back up slides

© 2026 Arm

ASIC industry trend - Resources

- 60-70 % of total project time is spent in verification
  - Depends on newly developed or existing pre-verified IPs in the design
- Need for productivity improvement
  - More demand for verification engineers
  - Design engineers spent half of their time doing verification.
  - Verification engineers spend more of their time debugging

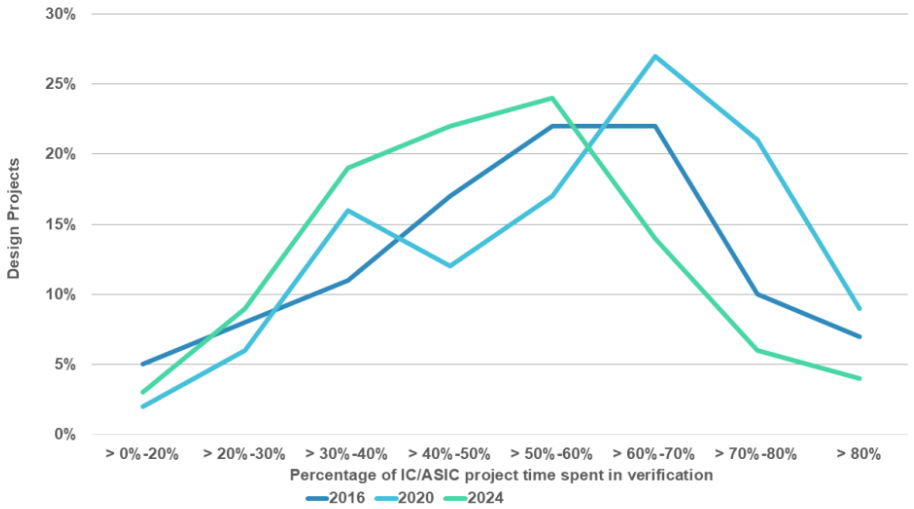


Figure 7. Percentage of IC/ASIC project time spent in verification.

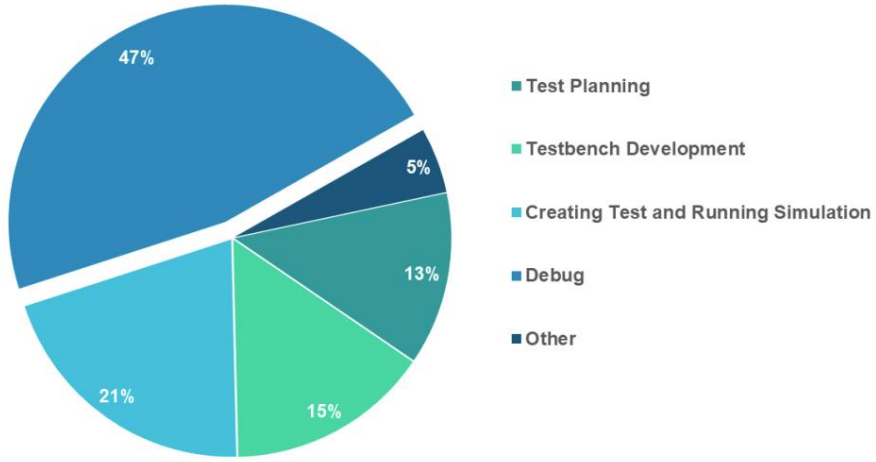


Figure 9. Where IC/ASIC verification engineers spend their time.

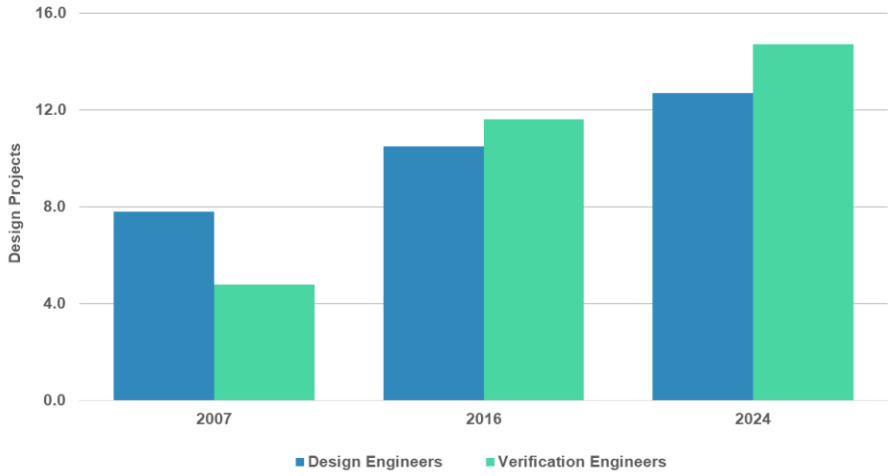


Figure 8. Mean peak number of IC/ASIC engineers.

© 2026 Arm

ASIC industry trend – Emulation and FPGA prototyping

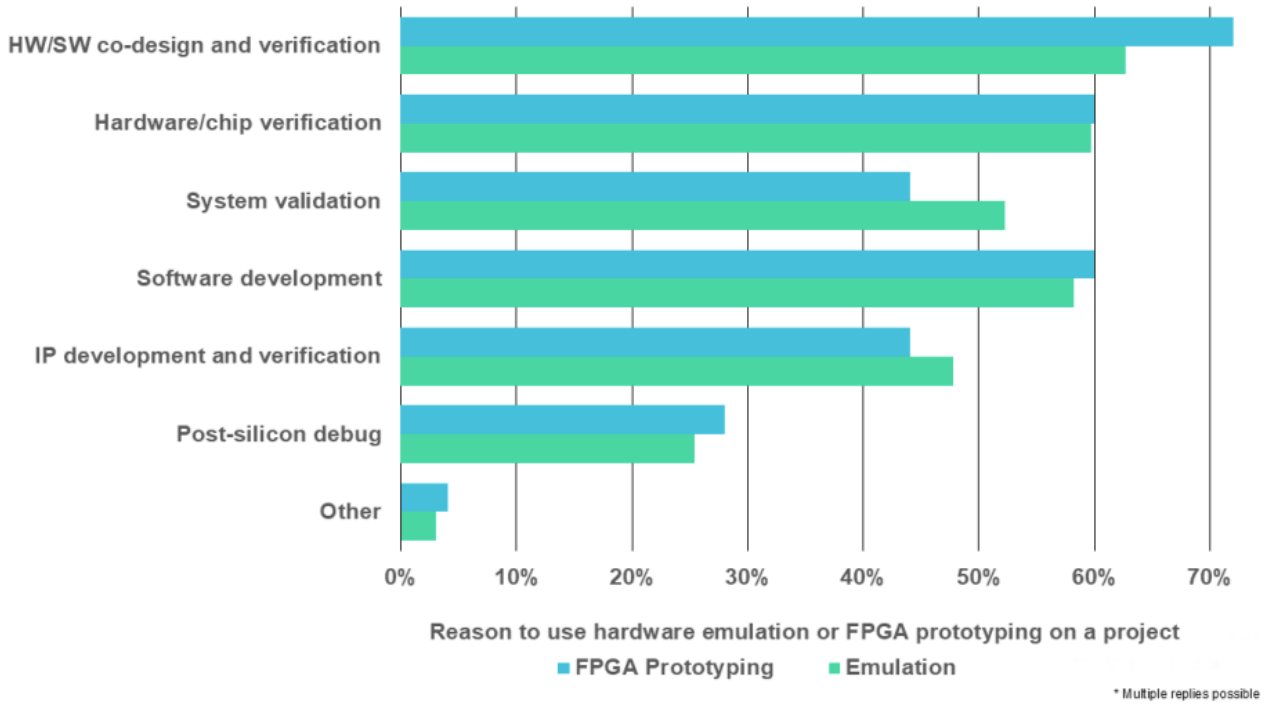


Figure 20. Why was emulation or FPGA prototyping used?

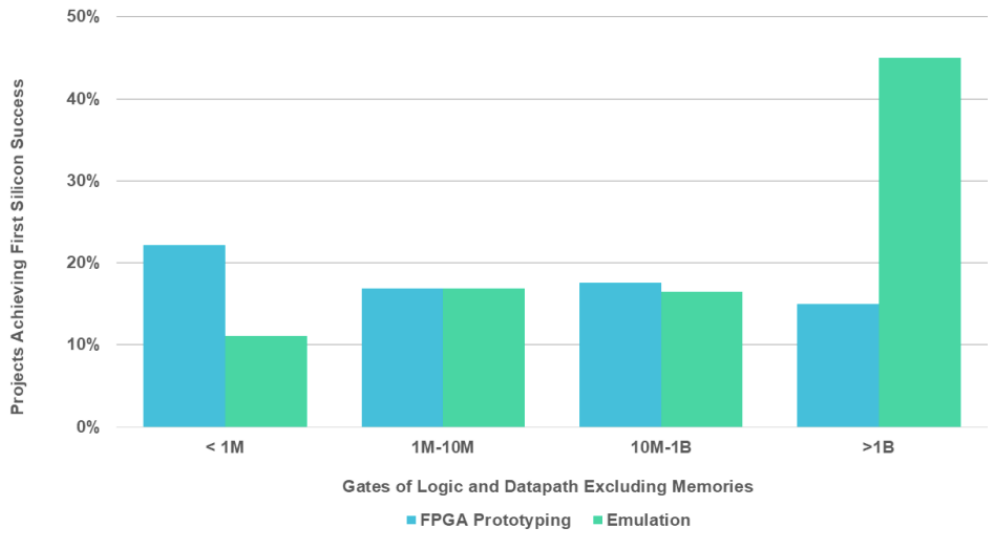


Figure 21. Emulation and FPGA prototyping adoption by design size.

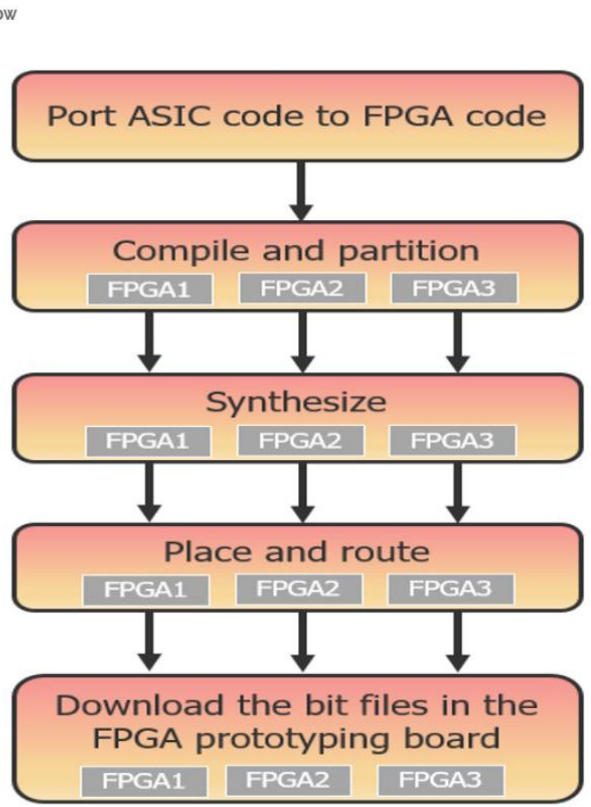
- Acceleration techniques are now essential to extend verification performance for many designs.
- Emulation and FPGA prototyping have emerged as key platforms for SoC integration verification, where both hardware and software are integrated into a system for the first time.
- Beyond SoC verification, these platforms are also widely used for software development

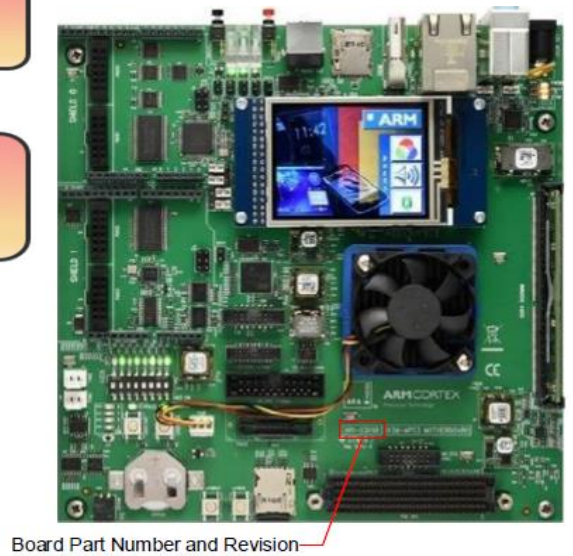
arm

© 2026 Arm

FPGA prototyping

- DUT is synthesized for the FPGA on the board
- Verify functionality of the system IC on FPGAs
  - HW+SW real use cases
  - Performance measurement
- Check all external interfaces are working correctly
- Test sequences (live data) are streamed to the board
- Monitored data is transferred to computer
- Key advantages
  - Hardware level verification possible
  - High speed, shortest time to market
  - Early software development and validation
- Main limitation
  - Very limited debuggability



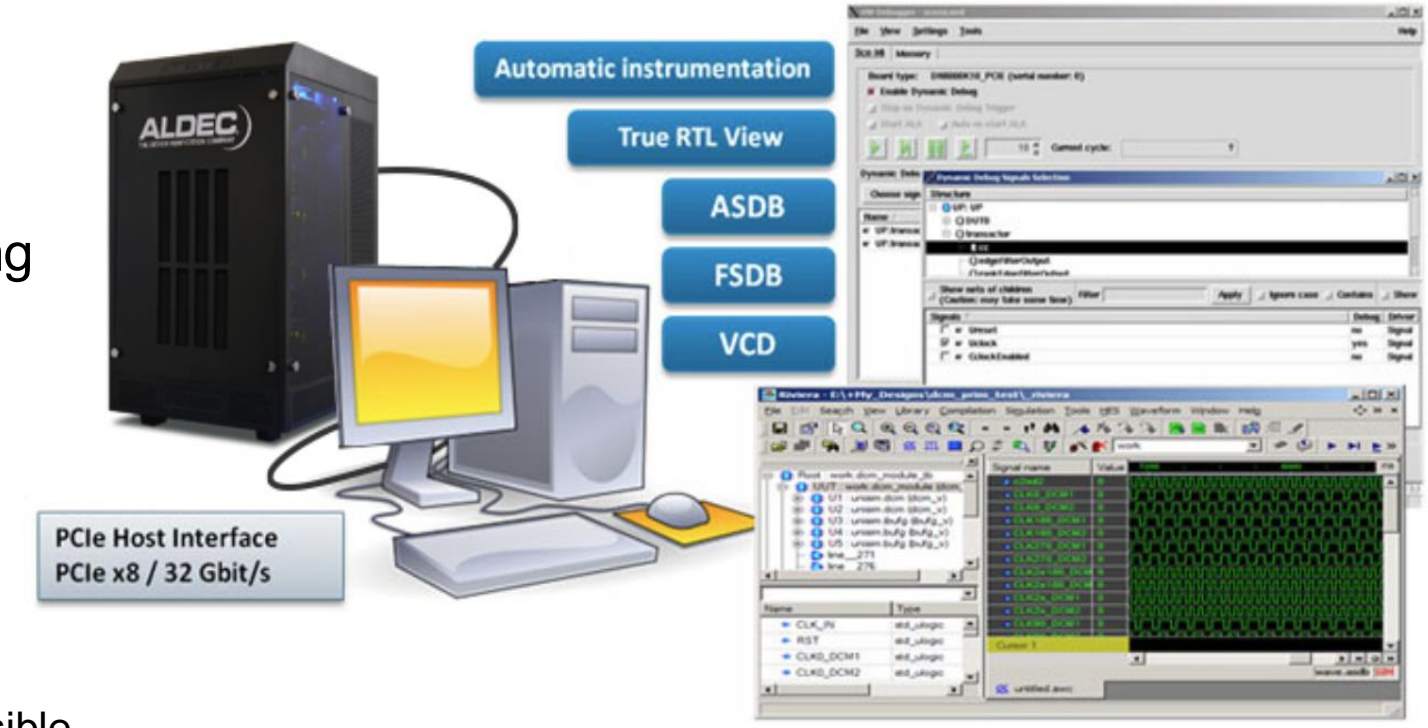


FPGA prototyping board

© 2026 Arm

Emulation

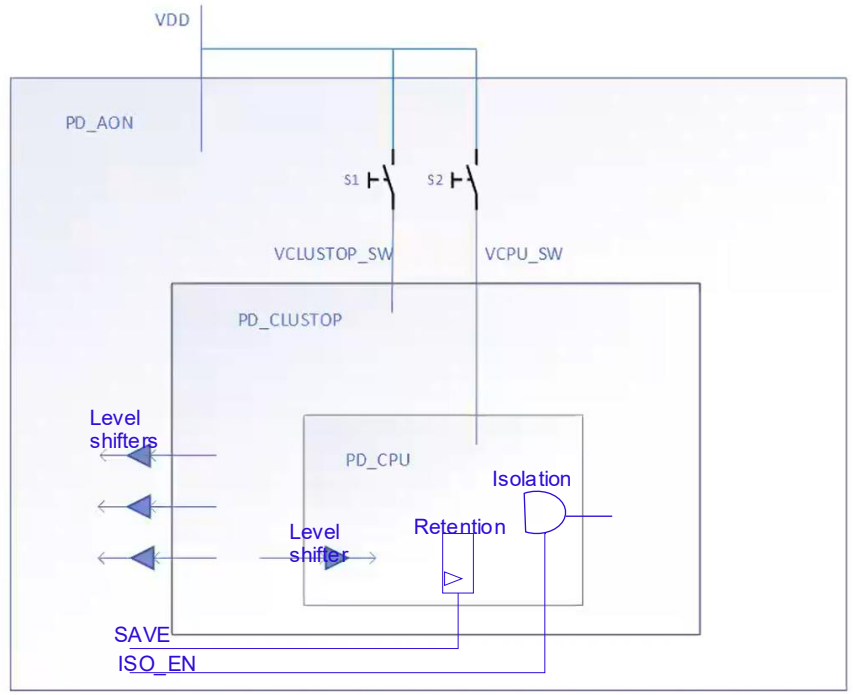
- DUT (system level) is loaded into the programmable emulator platform
  - Development and verification of embedded software
  - Verify system integration
- Multi-platform system
  - Simulations are done in emulator hardware
  - Testbenches provide input test vectors
- Workflow is similar to FPGA prototyping
- Key advantages
  - 100 to 10000 times faster than simulation
  - Higher visibility than FPGA prototyping
  - Can simulate multi-million gate design
- Main limitations
  - Expensive
  - Verification of interfaces is hard
  - Actual hardware level verification is not possible



© 2026 Arm

Power Aware Verification

- Verify the power intent and find bugs in the power architecture
  - May find bugs in the RTL e.g. related to control signals to isolation not happening at right time
- UPF – Unified Power Format
  - Static checks –
    - Tool performs analysis based on the UPF
    - structural checks related to isolations/shifters – missing/redundant/incorrect etc
  - Dynamic checks
    - Functional checking of UPF, rerun existing functional tests in power aware environment
    - Isolations - Clamp value check, isolation control signal corruption or on/off at right time
    - Level shifters - Missing level shifters or direction
    - Retention cells – retention not enabled or toggle during power off
    - Power related – input to a PD which is OFF toggles etc
- Power estimation
  - Power analysis can also be done in Power tool based on Gate Level netlist



PD – power domain, group of instances with same supply net

Level Shifter – interfaces with different voltage levels

Power switch – turn power on/off to a domain

Isolation – isolate power down domain from functional logic

Retention – specified registers maintained during power down

© 2026 Arm



The Arm trademarks featured in this presentation are registered trademarks or trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. All rights reserved. All other marks featured may be trademarks of their respective owners.

www.arm.com/company/policies/trademarks